Práctica. Bastiona tu web: cierra lo que te encontraron

Nombre y apellidos: _____

Lunes 24 de noviembre de 2026 · PPS · 4 horas · trabajo individual

El jueves pasado, en Hacking Ético, atacaste tu propia web y le encontraste unos cuantos fallos: inyección SQL, acceso a reservas de otros, enumeración de usuarios, falta de cabeceras de seguridad. Hoy los arreglas, uno a uno, y compruebas con una medida objetiva que cada arreglo funciona. Esto es bastionar: dejar el sistema igual de útil pero mucho más difícil de atacar.

La forma de trabajar de hoy es la de un profesional: **mides, arreglas y vuelves a medir**. Te doy un script que pasa seis comprobaciones a tu web y te dice en cuáles está bien y en cuáles mal. Lo lanzas al empezar (todo en rojo), arreglas, y al terminar tiene que estar todo en verde. Esa foto del antes y el después es lo que entregas.

1. Qué vas a aprender hoy

1. Cerrar una inyección SQL con consultas parametrizadas.
2. Impedir el acceso a datos de otro usuario comprobando la autorización.
3. Evitar el XSS dejando que la plantilla escape el contenido.
4. No filtrar qué usuarios existen y limitar los intentos de acceso.
5. Añadir cabeceras de seguridad y asegurar la cookie de sesión, en la aplicación y en nginx.
6. Comprobar con una medida objetiva que cada arreglo funciona.

2. Lo que necesitas

Tu web del 17 de noviembre, arrancada en tu Ubuntu (`sudo systemctl start miweb nginx`). Trabaja sobre esa misma carpeta. Del material de hoy (web del curso, PPS, Prácticas) descarga `comprobar.sh` y `nginx-bastionado.conf`.

3. Toma la foto del "antes" (15 minutos)

Con tu web arrancada, lanza el comprobador y **guarda su salida**:

```
bash comprobar.sh | tee antes.txt
```

Vas a ver varias líneas en `[MAL]`: la inyección SQL, la fuga por UNION, el acceso a reservas ajenas, la enumeración de usuarios y las cabeceras que faltan. Esa es la lista de tareas de hoy. No las arregles todavía; primero entiéndelas.

4. Cierra la inyección SQL (40 minutos)

Abre `miweb/__init__.py` y busca la función `buscar`. El problema está aquí:

```
consulta = ("SELECT id, aula, fecha, franja FROM reservas "
            f"WHERE usuario_id = {g.usuario_id} AND motivo LIKE '%{texto}%'")
filas = bd.conexion().execute(consulta).fetchall()
```

El texto que escribe el usuario se mete directamente dentro de la consulta con una *f-string*. Por eso una comilla la rompe: el usuario puede escribir trozos de SQL. El arreglo es **no construir la consulta pegando texto**, sino usar parámetros: escribes `?` donde va cada valor y le pasas los valores aparte. Así la base de datos trata lo que escribe el usuario como un dato, nunca como código:

```
    consulta = ("SELECT id, aula, fecha, franja FROM reservas "
                "WHERE usuario_id = ? AND motivo LIKE ?")
    filas = bd.conexion().execute(consulta, (g.usuario_id, f"%{texto}%")).fetchall()
```

Fíjate: los `%` del `LIKE` se quedan en el valor (`%texto%`), no en la consulta. Reinicia la web (`sudo systemctl restart miweb`) y vuelve a lanzar `comprobar.sh`: las dos primeras comprobaciones tienen que pasar a `[OK]`.

5. Protege las reservas ajenas (30 minutos)

En la misma función `ver_reserva`, ahora mismo se registra que alguien mira una reserva que no es suya, pero **se le enseña igual**. Eso es un IDOR: falta comprobar la autorización. Cambia ese trozo para que, si la reserva no es del usuario, no se muestre:

```
        if fila["usuario_id"] != g.usuario_id:
            registro.evento("reserva_ajena", 404,
                            detalle=f"intento id={reserva_id} propietario=
{fila['propietario']}")
            abort(404)
        return render_template("reserva.html", r=fila)
```

Devolvemos 404 (y no 403) a propósito: así no confirmamos siquiera que esa reserva existe. Reinicia y comprueba: el punto 3 pasa a `[OK]`.

6. Evita el XSS (20 minutos)

El jueves metiste un `<script>` en el motivo de una reserva y se ejecutaba. Abre `miweb/templates/reserva.html` y mira cómo se muestra el motivo:

```
<div class="motivo">{{ r.motivo|safe }}</div>
```

Ese `|safe` le dice a la plantilla "confía en este texto y no lo toques". Es justo lo que no hay que hacer con algo que escribe un usuario. Quítalo:

```
<div class="motivo">{{ r.motivo }}</div>
```

Sin el `|safe`, Jinja (el motor de plantillas) escapa el texto: un `<` se convierte en `<` y el navegador lo enseña como texto, no lo ejecuta. Vuelve a crear una reserva con un `<script>` en el motivo y ábrela: ahora tiene que verse el texto tal cual, sin saltar nada.

7. No chives los usuarios y limita los intentos (35 minutos)

Dos cosas que vio Hacking Ético: el login decía si el usuario existía, y dejaba probar contraseñas sin fin.

Primero, el mensaje. En la función `login`, junta los dos casos de fallo (usuario que no existe y contraseña mala) en una sola respuesta con **el mismo mensaje**, para no dar pistas:

```
            if fila is None or not check_password_hash(fila["hash"], clave):
                motivo = "usuario inexistente" if fila is None else "contraseña
incorrecta"
                registro.evento("login_fallido", 401, usuario=nombre,
detalle=motivo)
                flash("Usuario o contraseña incorrectos.")
                return render_template("login.html"), 401
```

Fíjate en un detalle importante: al usuario le damos el mismo mensaje, pero en el **registro** interno seguimos apuntando cuál fue el motivo. El que no debe saberlo es el atacante, no tu SOC.

Segundo, el límite de intentos. Añade un control sencillo que cuente los intentos por IP y bloquee cuando se pasan. Arriba del todo del fichero, junto a los `import`:

```
from collections import defaultdict
from time import monotonic

_intentos: dict[str, list[float]] = defaultdict(list)


def _demasiados_intentos(ip, limite=5, ventana=300.0):
    ahora = monotonic()
    _intentos[ip] = [t for t in _intentos[ip] if ahora - t < ventana]
    _intentos[ip].append(ahora)
    return len(_intentos[ip]) > limite
```

Y al principio del `POST` del login, antes de mirar la base de datos:

```
            if _demasiados_intentos(request.remote_addr or "-"):
                registro.evento("login_bloqueado", 429, usuario=nombre,
detalle="demasiados intentos")
                flash("Demasiados intentos. Espera unos minutos.")
                return render_template("login.html"), 429
```

Esto cuenta en memoria y se reinicia si reinicias la web; para el aula vale. En producción se guardaría en algo que sobreviva al reinicio y se combinaría con otras medidas, pero la idea es esta. Reinicia y comprueba el punto 4.

8. Cabeceras de seguridad y cookie (35 minutos)

Faltan las cabeceras que le dicen al navegador cómo protegerte. Las ponemos en **dos sitios**: en la aplicación (para que viajen siempre) y en nginx (defensa en profundidad).

En `miweb/__init__.py`, añade una función que las ponga en cada respuesta y asegura la cookie de sesión. Mete esto y llámalo desde `crear_app` (una línea `_seguridad(app)` antes de montar las rutas):

```
def _seguridad(app):
    @app.after_request
    def cabeceras(resp):
        resp.headers["X-Frame-Options"] = "DENY"
        resp.headers["X-Content-Type-Options"] = "nosniff"
        resp.headers["Referrer-Policy"] = "no-referrer"
        resp.headers["Content-Security-Policy"] = "default-src 'self'; frame-
ancestors 'none'"
        return resp
    app.config.update(
        SESSION_COOKIE_HTTPONLY=True,
        SESSION_COOKIE_SAMESITE="Lax",
    )
```

Cada cabecera hace algo concreto: `X-Frame-Options` impide que metan tu web en un marco de otra página (clickjacking), `X-Content-Type-Options` evita que el navegador adivine tipos de fichero, `Content-Security-Policy` limita de dónde se puede cargar código (tu mejor red contra el XSS que queda), y `Referrer-Policy` no filtra a dónde iba el usuario. La cookie con `HttpOnly` no la puede leer el JavaScript (otra barrera contra el XSS) y con `SameSite` no se manda desde otras webs (CSRF).

Ahora en nginx, para que las cabeceras estén aunque un día la app falle. Abre `nginx-bastionado.conf`, copia las líneas `add_header` dentro del bloque `server` de tu `/etc/nginx/sites-available/miweb`, y recarga:

```
sudo nginx -t && sudo systemctl reload nginx
```

Reinicia también la aplicación (`sudo systemctl restart miweb`) y comprueba los puntos 5 y 6.

9. Toma la foto del "después" (20 minutos)

Lanza otra vez el comprobador y guárdalo:

```
bash comprobar.sh | tee despues.txt
```

Tiene que estar **todo en `[OK]`**. Pon `antes.txt` y `despues.txt` uno al lado del otro y mira el cambio. Comprueba además que la web sigue funcionando para lo normal: entra, crea una reserva, búscala. Un bastionado que rompe la aplicación no sirve; hay que cerrar el fallo **sin** quitar la función.

Pasa también las pruebas del proyecto, que no se te haya roto nada:

```
cd ~/pps/PPS-2026-11-17-miweb-LAB   # tu carpeta del proyecto
. .venv/bin/activate
python -m pytest
```

10. Preguntas

1. ¿Por qué una consulta parametrizada evita la inyección y una f-string no?
2. ¿Por qué devolvemos 404 y no 403 cuando alguien pide una reserva que no es suya?
3. Has quitado un `|safe`. ¿Qué hacía y por qué era peligroso con datos de un usuario?
4. Al arreglar la enumeración, le damos al usuario el mismo mensaje pero en el registro seguimos apuntando el motivo real. ¿Por qué esa diferencia?
5. Has puesto las cabeceras en la aplicación y en nginx. ¿Por qué en los dos sitios?

11. Si algo no funciona

`comprobar.sh` dice que no conecta: tu web no está arrancada o no está en el 8080. Comprueba `sudo systemctl status miweb nginx` y que el comprobador apunta a `http://127.0.0.1:8080` (puedes cambiarlo con `WEB=... bash comprobar.sh`).

Después de un cambio, el comprobador sigue en `[MAL]`: ¿has reiniciado la aplicación (`sudo systemctl restart miweb`)? Los cambios en el código no se aplican hasta reiniciar. Si editaste nginx, además `sudo systemctl reload nginx`.

`sudo systemctl restart miweb` falla: mira el motivo con `sudo journalctl -u miweb -n 30 --no-pager`. Casi siempre es un error de Python que acabas de introducir (una indentación, un paréntesis). Lee la última línea del error.

Las pruebas (`pytest`) fallan después de bastionar: lee cuál falla. Si es una de funcionamiento, has roto algo (por ejemplo, el límite de intentos bloqueando una prueba que hace muchos logins); ajústalo. Si es una de las de secretos, repasa que no deshiciste lo del día 17.

`comprobar.sh` dice que la cookie ya era HttpOnly antes: es normal, Flask la pone así de serie; lo que faltaba era `SameSite`.

12. Qué tienes que entregar (20 minutos)

Carpeta `NN-PPS-bastionado` en ZIP con:

1. `antes.txt` y `despues.txt` (la foto del comprobador antes y después).
2. Tu `miweb/__init__.py` y `miweb/templates/reserva.html` ya bastionados.
3. El fragmento de tu configuración de nginx con las cabeceras.
4. La salida de `python -m pytest` en verde.
5. Las respuestas a las preguntas del apartado 10.

13. Antes de entregar, comprueba que

1. `despues.txt` está todo en `[OK]`.
2. La web sigue funcionando para lo normal (entrar, crear y buscar reservas).
3. `python -m pytest` pasa entero.
4. No has dejado ningún secreto en el código (sigue valiendo lo del día 17).
5. El registro de seguridad sigue apuntando los eventos (no lo has desactivado al tocar el login).

14. Seguridad y por qué importa

Hoy has cerrado tú mismo los agujeros que tú mismo encontraste. Ese ciclo —atacar, medir, arreglar, volver a medir— es exactamente lo que hace un equipo de seguridad: no basta con decir "ya está arreglado", hay que demostrarlo con una prueba que antes fallaba y ahora pasa. El miércoles, en Incidentes, cerramos el círculo: mirarás en tu SOC qué ataques de los del jueves habría parado este bastionado y cuáles habrían pasado igualmente, porque ninguna defensa es total. El jueves 26, en Hacking Ético, repetirás las pruebas sobre la web ya bastionada y verás, negro sobre blanco, qué ha cambiado.

Referencias

- Consultas parametrizadas (OWASP, hoja de inyección SQL): https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- Control de acceso (OWASP A01): https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- Escape automático en Jinja: https://jinja.palletsprojects.com/en/stable/templates/#html-escaping
- Cabeceras de seguridad HTTP (MDN): https://developer.mozilla.org/es/docs/Web/HTTP/Headers
- Content Security Policy (MDN): https://developer.mozilla.org/es/docs/Web/HTTP/CSP
- Atributos de las cookies (MDN): https://developer.mozilla.org/es/docs/Web/HTTP/Headers/Set-Cookie